Un semplice programma Java, formato da una sola classe, assume la seguente struttura generale:

```
class NomeClasse
{
   public static void main(String args[])
   {
      // dichiarazioni di variabili
      . . .
      // istruzioni
      . . .
   }
}
```

La classe dichiarata in questo modo contiene il solo metodo *main*. Questo metodo raggruppa le dichiarazioni di variabili e le istruzioni che compongono l'applicazione Java.

Esso viene dichiarato usando le parole chiave *public*, *static* e *void* che specificano alcune proprietà del metodo:

- **public** indica che il metodo è pubblico ed è visibile;
- **void** indica che non ci sono valori di ritorno;
- **static** indica che il metodo è associato alla classe e non può essere richiamato dai singoli oggetti della classe (il suo significato sarà presentato più in dettaglio nel seguito).

Dopo il nome del metodo, tra parentesi, sono indicati i parametri. Il metodo *main* possiede come parametro un array di stringhe (indicato con **args[]**) che corrisponde ai parametri passati dalla riga di comando quando viene eseguita l'applicazione.

Le parentesi graffe **{...}** sono usate per individuare un **blocco**, che può essere: una classe, un metodo oppure un insieme di istruzioni. All'interno di un blocco possono essere presenti altri blocchi in modo annidato. Per poter meglio distinguere questi blocchi conviene seguire e usare una tecnica di **indentazione**. Anche se un programma Java potrebbe essere scritto usando una sola riga, è utile disporre ordinatamente le varie istruzioni e gli elementi sintattici per migliorare la leggibilità del codice.

Le righe che iniziano con **//** indicano **frasi di commento**.

Solitamente si seguono le seguenti convenzioni:

- gli identificatori che si riferiscono al nome di attributi, di metodi e di oggetti cominciano con la prima lettera minuscola (per esempio, *nome*, *voto*, *aggiorna*, *auto*);
- gli identificatori che specificano il nome delle classi iniziano con la lettera maiuscola (per esempio, *Libro*, *Automobile*, *Rettangolo*);
- se gli identificatori sono formati da più parole, queste vengono unite e ogni parola successiva alla prima ha l'iniziale maiuscola (per esempio, *totFattura*, *contaSecondi*, *calcolaMedia*).

Le **parole chiave** sono un insieme di parole riservate di Java che non possono essere usate come identificatori.

Parole chiave del linguaggio Java

abstract	continue	goto	package	this
assert	default	if	private	throw
boolean	do	implements	protected	throws
break	double	import	public	transient
byte	else	instanceof	return	try
case	enum	int	short	void
catch	extends	interface	static	volatile
char	final	long	super	while
class	finally	native	switch	
const	for	new	synchronized	

I programmi Java devono essere scritti seguendo le regole dalla **sintassi** del linguaggio che specifica come combinare tra loro le parole chiave e gli identificatori in modo corretto.
Le regole della sintassi Java saranno illustrate nei prossimi paragrafi.

5 Variabili e costanti

Le **variabili** sono i contenitori con i quali è possibile memorizzare i dati. Le variabili possono cambiare valore durante l'esecuzione del programma. Gli elementi che caratterizzano una variabile sono il *tipo*, il *nome* e la sua *visibilità*.

In Java la dichiarazione di una variabile assume la seguente forma:

tipo nomevariabile;

Se, per esempio, si vuole dichiarare una variabile che deve contenere un valore numerico, si deve scrivere:

```
int prezzo;
```

Durante la dichiarazione delle variabili è possibile inserire il valore iniziale da assegnare ad esse. L'**assegnamento** di un valore di inizializzazione a una variabile si ottiene usando il segno uguale (=).

tipo nomevariabile = valoreiniziale;

Per esempio:

```
double altezza = 1.83;
```

Una **costante**, a differenza delle variabili, può assumere un solo valore durante tutta l'esecuzione del programma.
Per dichiarare una costante, bisogna far precedere alla dichiarazione la parola chiave **final**.
Il valore di conversione tra la *yard* (0.914 metri) e il *metro* può essere espresso con la costante

```
final double YARD_METRO = 0.914;
```

6 Tipi di dato

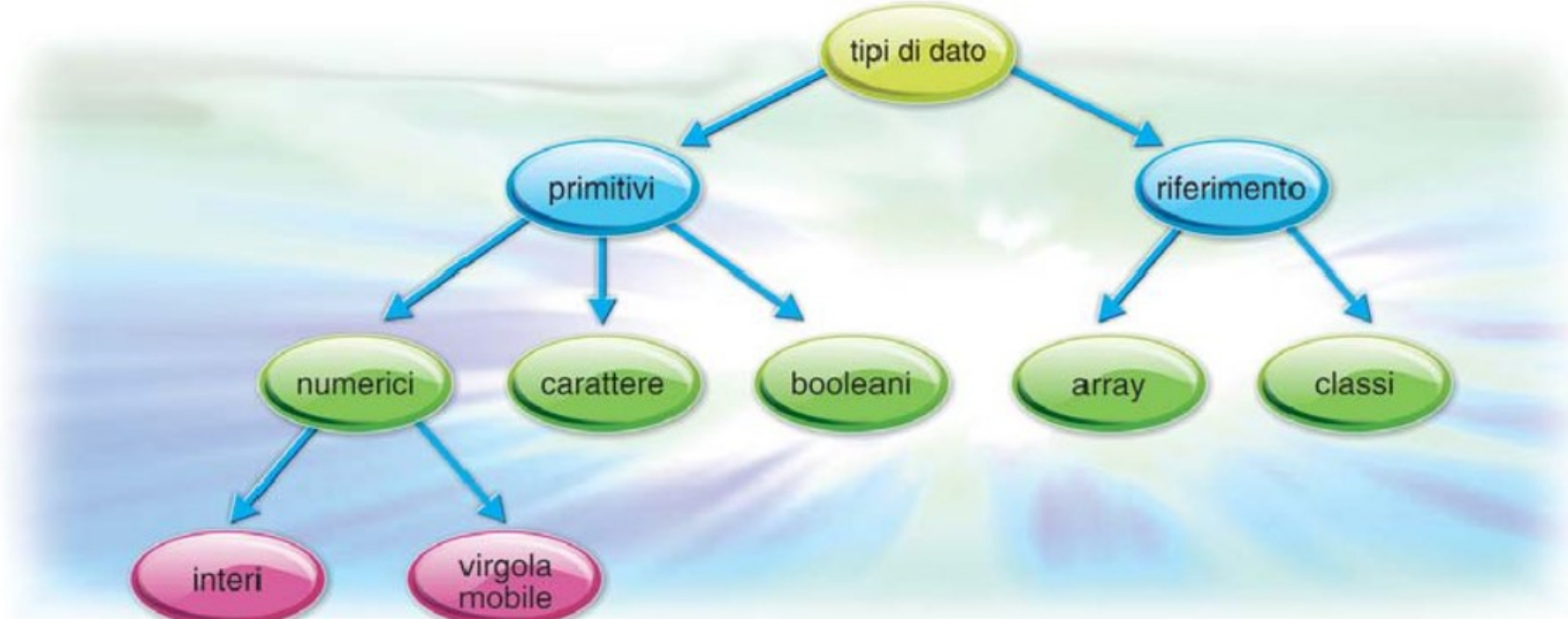
Il **tipo di dato** stabilisce qual è l'insieme di valori e operazioni accettabili da una variabile. In Java si distinguono due tipi di dato:

- tipi primitivi
- tipi riferimento.

I tipi di dato **primitivi** sono analoghi a quelli offerti da tutti i linguaggi di programmazione. Possono essere raggruppati in tre categorie: numerici, carattere e booleani.

I tipi di dato **riferimento** sono gli array e le classi, che saranno presentati più avanti.

Schematizzando la classificazione dei tipi di dato presenti in Java, si ottiene la seguente rappresentazione gerarchica.



8 Operatori

Gli **operatori** sono i caratteri speciali che identificano particolari operazioni sui dati. Per esempio, si possono usare gli operatori matematici per le operazioni tra i numeri oppure l'operatore di concatenazione tra le stringhe.

Operatori del linguaggio Java

=	>	<	!	~	?:					
==	<=	>=	!=	&&	||	++	--			
+	-	*	/	&	|	^	%	<<	>>	>>>
+=	-=	*=	/=	&=	|=	^=	%=	<<=	>>=	>>>=

Le **operazioni aritmetiche** di base possono essere eseguite con i quattro operatori fondamentali: **+**, **-**, *****, **/**.
L'operatore **%** è usato per calcolare il **resto** della divisione tra due numeri.

Per le operazioni di **input** esiste un oggetto analogo **System.in** che gestisce il flusso di dati inseriti da tastiera. L'oggetto *System.in*, in pratica, viene mascherato con altri oggetti più potenti che forniscono maggiori funzionalità.

Si utilizza la classe **BufferedReader** nel seguente modo:

```
InputStreamReader input = new InputStreamReader(System.in);
BufferedReader tastiera = new BufferedReader(input);
```

Con queste dichiarazioni viene definito un oggetto *tastiera*, di classe *BufferedReader*, usando l'operatore **new** che crea un nuovo oggetto, come verrà spiegato più in dettaglio nel prossimo capitolo.
La classe *BufferedReader* mette a disposizione il metodo **readLine** che consente di leggere, in questo caso da *standard input*, una riga per volta.
Una linea viene considerata terminata quando viene premuto il tasto di *Invio*.
Questo metodo restituisce solo stringhe, quindi se si vogliono acquisire valori numerici, si deve effettuare la conversione tra stringhe e numeri.

Per effettuare la **lettura di una stringa** dobbiamo dichiarare una variabile stringa e poi usare il metodo *readLine* con l'oggetto *tastiera* che abbiamo definito in precedenza.

```
String nome;
nome = tastiera.readLine();
```

L'operazione deve essere completata considerando le eccezioni che possono essere generate dal metodo *readLine*. L'**eccezione** segnala una situazione anomala. Quando si verifica un'eccezione bisogna prevedere un blocco di istruzioni per gestire questa situazione. Il blocco dell'eccezione è realizzato con il costrutto **try { } catch { }**.

Quindi la lettura di una stringa si effettua correttamente nel seguente modo:

```
String nome;
try
{
  nome = tastiera.readLine();
}
catch(Exception e) {}
```

Il segmento di codice stabilisce che, al verificarsi dell'eccezione, non deve essere effettuata alcuna operazione.

Per effettuare la lettura di numeri, occorre convertire la stringa ottenuta dall'operazione di input nel seguente modo:

```
String leggiNumero;
int num;

try
{
  leggiNumero = tastiera.readLine();
  num = Integer.valueOf(leggiNumero).intValue();
}
catch(Exception e)
{
  System.out.println("\nNumero non corretto!");
  return;
}
```

L'istruzione aggiuntiva consente di convertire una stringa in un numero intero, attraverso due metodi della classe **Integer**: uno (*valueOf*) converte il parametro stringa in un oggetto di classe *Integer*, l'altro metodo (*intValue*) restituisce un valore intero.

In questo caso, l'eccezione che può essere generata si verifica quando i caratteri inseriti non corrispondono a cifre numeriche. Se per esempio si inseriscono i valori 34f5 oppure 45.45, essi non vengono riconosciuti come interi e quindi provocano un'eccezione che causa la terminazione del programma.

Per leggere un numero a virgola mobile, occorre cambiare il modo di conversione.

Per esempio, se si vuole ottenere un valore *float*, si deve scrivere:

```
numFloat = Float.valueOf(leggiNumero).floatValue();
```

In modo del tutto analogo, per ottenere un valore *double*:

```
numDouble = Double.valueOf(leggiNumero).doubleValue();
```